

Assignment 1 — Web Development Fundamentals

1. Frontend, Backend, and Full-Stack Development

Frontend Development: This covers everything a visitor directly sees and interacts with on a website — layouts, buttons, images, menus, forms, and animations. It's built primarily with HTML, CSS, and JavaScript.

Example: On Flipkart or Amazon, every visual element you browse — the search bar, product cards, images — is rendered by the frontend.

Backend Development: This is the server-side logic that users never see directly. It handles incoming requests, manages databases, authenticates users, and runs business logic. Common backend stacks include Node.js, Java, Python, PHP, and MySQL.

Example: Clicking "Buy Now" triggers backend processes that verify payment, update inventory, and record the order.

Full-Stack Development: A full-stack developer works across both layers, capable of building an entire application — from the user interface down to the database — independently.

2. The Client-Server Model

Client (Browser) → Internet → Server → Browser

The browser acts as the client and sends a request over the internet to a web server. The server processes that request and returns the required resources (HTML, CSS, JavaScript, images, or data), which the browser then renders as a webpage.

3. Steps a Browser Follows to Load a Webpage

1. The user types a URL into the addressbar.
2. A DNS lookup resolves the domain name to the server's IP address.
3. The browser sends an HTTP (or HTTPS) request to that server.
4. The server processes the request on its end.
5. The server responds with HTML, CSS, JavaScript, and any other required assets.
6. The browser parses these files, constructs the page, and displays it to the user.

4. Setting Up a Development Environment

VS Code: A code editor used to write and manage source files.

Web Browser: Used to run and test the website.

Live Server Extension: Automatically reloads the browser whenever a file is saved.

Node.js: A JavaScript runtime that runs code outside the browser and manages packages via npm.

Git & GitHub: Used for version control and hosting project repositories online.

5. What Is a Web Server?

A web server is software (often paired with dedicated hardware) that hosts a website's files and delivers them to users on request. Well-known examples are Apache, Nginx, and Microsoft IIS.

6. Common Web Development Roles

Frontend Developer: Focuses on building and designing the user-facing interface.

Backend Developer: Builds server-side logic, APIs, authentication systems, and database connections.

Database Administrator (DBA): Responsible for setting up, securing, tuning, and backing up databases.

7. Configuring VS Code for Web Development

Install VS Code, then add the Live Server extension from the Extensions marketplace. Set up a project folder with index.html, style.css, and script.js inside it. Open that folder in VS Code, then click "Go Live" to preview the site. (Attach a screenshot of your own setup here.)

8. Static Websites vs Dynamic Websites

Static website: Serves identical, fixed content to every visitor. Example: a personal portfolio site.

Dynamic website: Generates content that varies based on the user or on data pulled from a database. Examples: Amazon, Facebook, Instagram.

9. Web Browsers and Rendering Engines

Popular browsers include Google Chrome (Blink engine), Microsoft Edge (Blink engine), Mozilla Firefox (Gecko engine), Safari (WebKit engine), and Opera (Blink engine). A rendering engine is the component that translates HTML, CSS, and JavaScript into the visual page a user sees.

10. Overview of Web Architecture

Client → Web Server → Application/Backend → Database ↔ APIs → Response → Client

The browser (client) sends a request to the web server, which passes it to the backend application. The backend retrieves or updates data via the database and/or external APIs, then sends the resulting response back through the server to the client's browser.